Flavor Find: Consumer Backend Architecture and Sprint Plan

Stage 2 | Built on Engine v1.0 | October 2026 | Nairobi, Kenya | Confidential

1. Purpose and Ground Rules

The consumer backend (repo `ff-consumer`) serves the Flavor Find website and Android app. It owns everything about **people** (accounts, pantries, plans, saved recipes, subscriptions). The engine owns everything about **food**. The two meet only at the engine's public API.

Rule	What it means
One door to food	The backend never reads the engine database. It calls `/v1` with its own key (scopes `read:search`, `read:catalog`) and, for community recipes only, an internal key (`internal:curation`).
Engine stays user-free	The engine logs what was asked, never who asked. The backend therefore never sends user IDs, emails or device data to the engine.
Store IDs, not copies	Saved items keep engine `slug` and `dish_id` plus the `dataset_version` seen. Names and steps are fetched or cached, never owned.
Never invent food	Search, pairings, substitutions, missing items, diet and allergen answers all come from the engine. The backend adds personalisation and money, not food facts.
Same standards as the engine	Same CI/CD gates, mandatory 2FA for staff, signed images, audit log, backups and runbooks.

2. Scope

In scope (Stage 2)	Out of scope
Users, auth, profiles, dietary profiles	LLM chat (AI service, Stage 4)
Pantry, cook-with-what-I-have search	Hotel platform and tenant databases
Saved recipes, history, offline sync	Engine data changes (done through the engine's admin)
Meal plans, shopping lists, budget mode	Image or video hosting beyond avatars
Taste of 47 counties content	Native iOS app
Community submissions and reviews	Affiliate checkout (hooks only)
Flavor Find Plus subscription (M-Pesa)	Machine-learning recommendations
Notifications, sponsored content slots	

3. System Context

```
Website / Android app  ->  Caddy (TLS, WAF)  ->  Consumer API (Django + DRF)
                                                   |-- Services: pantry,
plans, shopping, budget, community, billing
                                                   |-- Engine client
(generated from engine OpenAPI)  ->  Recipe Engine /v1
                                                   |-- PostgreSQL (own DB)
Redis (cache, limits, queue)
                                                   |-- Celery worker + beat
(sync, notifications, rollups)
Staff (moderators, admins)  ->  Admin app (separate address, 2FA, VPN or
allowlist)
```

Same stack as the engine (Django, DRF, PostgreSQL, Redis, Celery, Docker, Caddy) so one team, one toolchain, one set of runbooks.

4. Django Apps

App	Responsibility
`core`	Base models (UUID, timestamps, soft delete), pagination, error format, permissions
`accounts`	Users, login methods, sessions, devices, consent, deletion and export
`profiles`	Language (en, sw), family size, diet tags, allergen exclusions, region preference, budget
`engine_client`	Typed client, retries, timeouts, circuit breaker, dataset-version cache, ambiguity helper
`pantry`	Pantry items (engine ingredient IDs), staples overrides, "I do not have oil"
`library`	Saved recipes, collections, cooking history, notes
`planner`	Weekly meal plans, servings, shopping lists, WhatsApp share text
`budget`	Budget mode on top of engine cost estimates, always labelled as estimates
`content`	Taste of 47 counties: stories, photos and editorial, keyed to engine region slugs
`community`	Recipe submissions (drafts), status tracking, contributor credit, reviews and ratings
`billing`	Plans, Flavor Find Plus, M-Pesa Daraja STK push, receipts, entitlements
`notifications`	Push, email, in-app; preferences; quiet hours
`partners`	Sponsored slots and affiliate hooks, clearly labelled
`sync`	Offline change queue for the Android app, conflict rules
`moderation`	Review queue for submissions and reviews, audit trail

App	Responsibility
`api`	Versioned routers (`/v1` public, `/internal` staff), OpenAPI schema

5. Data Model (own database)

Identity rules copied from the engine: UUID plus stable slug, status on everything, soft delete, change log.

Table	Key fields
`user`	id, email, phone, password_hash (Argon2id), language, status, created_at, deleted_at
`auth_identity`	user_id, provider (password, phone_otp, google), verified_at
`device`	user_id, platform, push_token, last_seen
`consent`	user_id, type (terms, privacy, marketing), version, granted_at
`profile`	user_id, household_size, region_slug, budget_kes_per_meal
`profile_diet`	user_id, diet_tag_slug, excluded_ingredient_ids[]
`pantry_item`	user_id, ingredient_id, added_at, expires_on (nullable)
`saved_recipe`	user_id, dish_id, dish_slug, dataset_version, collection_id, note
`cook_log`	user_id, dish_id, cooked_at, rating (nullable)
`meal_plan` / `meal_plan_entry`	plan(user_id, week_start); entry(plan_id, day, meal_type, dish_id, servings)
`shopping_list` / `shopping_item`	list(user_id, plan_id); item(ingredient_id, quantity_note, checked, price_estimate_kes)
`story`	id, slug, region_slug, title_en, title_sw, body, status, author

Table	Key fields
`submission`	id, user_id, payload JSON, engine_submission_id, status (draft, sent, in_review, accepted, rejected), reason
`review`	user_id, dish_id, stars, text, status
`plan` / `subscription`	plan(code, price_kes, features); subscription(user_id, plan_id, status, renews_at)
`payment`	user_id, provider_ref, amount_kes, status, receipt_no
`sponsored_slot`	partner, dish_id or ingredient_id, label, start, end
`engine_cache`	key (input hash + dataset_version), response, fetched_at
`change_log`, `audit_event`	actor, object, before/after, reason, created_at

No food tables. If it describes food, it lives in the engine.

6. Engine Integration Contract

Engine capability	Consumer feature	Handling
`POST /search/by-ingredients` (strict, explore, diet, exclude)	Cook with what I have	Send pantry words or IDs plus profile diet and exclusions. Show exact, can make, almost (explore only) with matched, missing, unused and score reasons
`POST /ingredients/resolve`	Typing and voice input	If `ambiguous`, show choices and never guess ("unga", "pepper"). Show "corrected spelling" notes
`GET /ingredients?q=`	Autocomplete	Cache, debounce on client
`GET /dishes`, `/dishes/{id}`	Recipe pages, Taste of 47 counties	Region, meal type and diet filters; `lang` en or sw

Engine capability	Consumer feature	Handling
`GET /dishes/{id}/pairings`	"Add a drink or side"	Show by type and strength
`GET /dishes/{id}/substitutions`	Diaspora mode, "what can I use instead"	Show quality grade, warn on emergency
`POST /dishes/{id}/missing`	Shopping gap, one-tap list	Creates shopping items
Cost and nutrition estimates	Budget mode, Plus nutrition	Always labelled estimate; filter by the user's budget in the backend
`GET /meta/version`, `/health`	Cache keys, status	Pin and report `dataset_version`
`POST /internal/submissions`	Community recipes	Drafts only, internal key, from the moderation worker

Behaviour rules

- **Safety first:** diet tags and allergen exclusions are always sent on every search; a profile with exclusions can never run an unfiltered search. Allergen text carries the standard reminder to confirm ingredients.
- **Caching:** key = input hash + `dataset_version`; cleared when the version changes. Dish detail is cached with ETag.
- **Resilience:** timeouts, one retry, circuit breaker. On `429`, honour `Retry-After`. If the engine is down, serve cached results and saved recipes, and say so in the app.
- **Contract safety:** the engine client is generated from the engine OpenAPI file; CI fails on any breaking change. Engine v1 is additive only, so the backend must ignore unknown fields.
- **Quantities:** the engine ignores quantities in v1. The backend keeps them only as notes on shopping items.
- **Dataset drift:** saved recipes store `dataset_version`; if a dish was changed or removed, show the current version and flag the change.

7. Consumer API (v1)

Area	Endpoints
Auth	`POST /auth/register`, `/auth/login`, `/auth/otp/verify`, `/auth/refresh`, `/auth/logout`, `DELETE /me`, `GET /me/export`
Profile	`GET/PATCH /me/profile`, `/me/diet`
Pantry and search	`GET/POST/DELETE /pantry`, `POST /search` (wraps the engine), `POST /resolve`
Library	`GET/POST/DELETE /saved`, `/collections`, `POST /cooked`
Planner	`GET/PUT /plans/{week}`, `POST /plans/{id}/shopping-list`, `GET /shopping-lists/{id}/share`
Budget	`POST /budget/suggest` (budget in KES, servings, diet)
Content	`GET /regions`, `/stories`, `/stories/{slug}`
Community	`POST /submissions`, `GET /submissions`, `POST /dishes/{id}/reviews`
Billing	`GET /plans`, `POST /subscribe` (STK push), `GET /subscription`, `POST /mpesa/callback`
Sync	`POST /sync/push`, `GET /sync/pull?since=`
Devices and meta	`POST /devices`, `GET /meta/version`, `GET /health`

Conventions match the engine: `/v1` path, additive changes only, six-month deprecation notice, cursor pagination, one JSON error shape with request ID, `lang` en or sw with English fallback, OpenAPI via drf-spectacular with generated TypeScript and Kotlin clients.

8. Security

The consumer backend holds **real personal data**, so it carries obligations the engine does not.

Area	Decision
Staff accounts	Same as engine: mandatory 2FA (authenticator or security key; no SMS or email codes), step-up for sensitive acts, admin on a separate address behind VPN or allowlist, session limits
End-user login	Password (Argon2id, breach check) or phone OTP; optional authenticator-app 2FA for users; short-lived access tokens plus rotating refresh tokens; device list and remote logout
Engine key	In the secret manager, separate test and live keys, rotated every 90 days with overlap, never in the app or client code
Payments	M-Pesa callback verified by source and signature, idempotent, amounts re-checked server side; no card data stored
Authorisation	Every row is scoped to its owner; automated tests try to read other users′ data
Abuse	Rate limits per user, IP and endpoint; OTP throttling; spam checks on submissions and reviews
Data protection	Kenya Data Protection Act, 2019: register if required, consent records, privacy notice, data export and deletion endpoints, breach procedure (regulator notice generally within 72 hours; confirm with a lawyer), minimal retention
Encryption	TLS everywhere, provider encryption at rest, separate backup key, field-level encryption for phone numbers

Area	Decision
Supply chain	Pinned dependencies, SBOM, signed images, Actions pinned by commit, OIDC, protected workflow files
Audit and alerts	Append-only audit log shipped externally; alerts on login anomalies, failed OTP spikes, payment callback failures, engine errors, backup failure

Retention starting points: request logs 90 days, audit logs 2 years, backups 35 days rolling, deleted accounts purged after a short grace period.

9. CI/CD, Environments and Releases

Reuse the engine's templates, rulesets and pipelines.

Item	Plan
Repo	`ff-consumer`, branch protection on main, signed commits, CODEOWNERS (auth, billing and workflows owned by the security reviewer), no bypass
PR pipeline (under 10 min)	ruff, mypy, gitleaks, bandit and semgrep, pip-audit, pytest with real Postgres and Redis, coverage (85% overall, 95% for `accounts`, `billing`, `engine_client`), migration checks, API contract (oasdiff, Schemathesis), image build and Trivy scan, SBOM
Engine contract job	Runs the backend against the engine **staging** (or a recorded mock) and fails if the engine OpenAPI changed in a breaking way
Main pipeline	Build once, sign, deploy to staging, smoke tests (health, login, search, save, plan, one sandbox payment), short perf check
Release	Signed tag, approval by a second person, backup first, expand-only migrations, rolling deploy, health gate with auto-rollback, 30-minute watch

Item	Plan
Nightly and weekly	Fuzzing, DAST, backup integrity, restore drill, load test, access review
Environments	Local (docker-compose with a mock engine), CI, staging (M-Pesa sandbox), production
Mobile compatibility	Old Android app versions stay supported by API versioning; a minimum supported app version flag forces upgrades only when unavoidable

10. Testing

Layer	Checks
Unit	Services, budget logic, sync conflict rules, entitlements
Property-based	Sync is idempotent; a plan never loses entries; entitlement never grants Plus without a paid or trial subscription
Permission tests	No user can read or change another user's data
Engine contract	Generated client compiles; ambiguity, zero-result, 429 and engine-down paths tested
Safety golden set (about 40 cases)	Vegetarian profile never returns meat dishes; excluded allergen never appears; Swahili input resolves; explore mode labels missing items; "unga" asks, never guesses
Payments	Duplicate callbacks, timeouts, wrong amounts, expired subscriptions
Performance	Search through the backend p95 under 250 ms (engine under 150 ms plus overhead) at 50 requests per second; query-count budgets
Offline	Android sync replay with conflicting edits

11. Observability and Targets

Sentry, structured logs with request IDs, uptime checks on health and a real search, dashboards for latency, errors, engine call health and cache hit rate, signups, daily active users, searches per user, zero-result rate, saves, plan creation, payment success and churn.

Objective	Target
Availability	99.5% per month
Search p95	Under 250 ms
Error rate	Under 0.5%
Payment confirmation	Under 30 seconds for 95%
Recovery	RPO 1 hour, RTO 4 hours

12. Sprint Plan

Five two-week iterations (about 10 weeks, one to two developers; estimates to confirm in Iteration 1). Each iteration ships to staging and runs the engine contract tests.

Iteration	Build	Security, CI/CD and ops	Exit check
1. Foundations (wks 1 to 2)	Repo from the engine template, Django project, core and accounts, register and login, profile and diet, generated engine client with key, health and version pages	Org rules reused, pre-commit, CI skeleton, staging with TLS, secret manager, staff 2FA and roles, admin on its own address	Register, log in, set a diet, call the engine from staging and see its `dataset_version`; first rollback practised

Iteration	Build	Security, CI/CD and ops	Exit check
2. Search and pantry (wks 3 to 4)	Pantry, resolve and autocomplete, search wrapper with strict and explore, ambiguity flow, engine cache, circuit breaker, pairings, substitutions, missing-items	Full PR pipeline, engine contract job, rate limits, permission tests	Pantry search returns tiers with reasons; "unga" asks; vegetarian and allergen cases pass; engine-down fallback works
3. Library, planner, offline (wks 5 to 6)	Saved recipes, collections, cook log, meal plans, shopping lists, WhatsApp share, sync push and pull, content and regions (Taste of 47 counties)	Audit log wired and shipped, nightly DAST and fuzzing, OpenAPI docs and TypeScript and Kotlin clients	Plan a week, generate a shopping list, edit offline on two devices and sync without loss
4. Community and money (wks 7 to 8)	Submissions to the engine as drafts, moderation queue, reviews, budget mode, Plus plans, M-Pesa STK push and callback, entitlements, notifications, sponsored slots	Payment tests and sandbox drills, step-up 2FA for staff, data export and deletion, consent records, threat-model review	Pay with sandbox M-Pesa and unlock Plus; submission reaches the engine as a draft; deletion removes a user
5. Hardening and release (wks 9 to 10)	Bug fixing, tuning, final docs and runbooks, store-ready API freeze for the app and website	Load and soak tests, external penetration test and fixes, backup restore and rollback drills, incident tabletop, go-live checklist	Exit criteria below met; v1.0.0 live; website and Android app integrate

Suggested parallel tracks: backend (above), content (stories and region copy in `content`), and app teams consuming the generated clients from Iteration 2.

13. Exit Criteria for the Consumer Backend v1.0

Area	Criterion
Engine integration	Every engine endpoint listed in section 6 used correctly; contract job green; ambiguity, 429 and outage paths proven
Safety	Safety golden set at 100% for diet and allergen cases; zero unfiltered searches for profiles with exclusions
Product	Search, pantry, saved recipes, planner, shopping lists, budget mode, community submissions, Plus billing and offline sync all working end to end
Quality	Coverage gates met; property and permission tests green
Performance	Search p95 under 250 ms at 50 requests per second on staging
Security	2FA enforced on every staff account; no open high or critical findings; penetration test fixed; go-live checklist signed
Privacy	Consent, export, deletion, privacy notice and breach procedure in place
Ops	Signed images only, rollback and restore drills passed, dashboards and alerts live, runbooks complete
Docs	OpenAPI complete; app teams integrate in under a day

14. Decisions to Make in Week 1

- Login methods at launch: phone OTP, email and password, Google, or all three (recommended: phone OTP plus email).
- Does Free tier gate search, or only AI, plans and offline (recommended: search stays free; Plus gates plans, nutrition, offline and later AI)?
- Plus price (Master Plan suggests KES 199 to 299 per month) and trial length.
- Who moderates community submissions and reviews, and with what turnaround.
- Who writes the Taste of 47 counties stories, and which counties launch first.
- Whether shopping-list prices come only from engine estimates (recommended for v1) or a separate price source later.
- Cloud provider and region (match the engine for low latency).
- Whether the consumer backend and the engine share one team or two.

Next step: approve this document, create `ff-consumer` from the engine template, and start Iteration 1. I can produce the Iteration 1 tickets and the OpenAPI skeleton for the consumer API next.